Controls what happens to the associated object when its owner is destroyed:

- `:destroy` causes the associated object to also be destroyed
- `:delete` causes the associated object to be deleted directly from the database (so callbacks will not execute)
- `:nullify` causes the foreign key to be set to `NULL`. Callbacks are not executed.
- `:restrict_with_exception` causes an exception to be raised if there is an associated record
- `:restrict_with_error` causes an error to be added to the owner if there is an associated object

It's necessary not to set or leave `:nullify` option for those associations that have `NOT NULL`database constraints. If you don't set `dependent` to destroy such associations you won't be able to change the associated object because initial associated object foreign key will be set to unallowed `NULL` value.

4.2.2.5 `:foreign_key`

By convention, Rails assumes that the column used to hold the foreign key on the other model is the name of this model with the suffix `_id` added. The `:foreign_key` option lets you set the name of the foreign key directly:

```
class Supplier < ActiveRecord::Base
  has_one :account, foreign_key: "supp_id"
end
```

In any case, Rails will not create foreign key columns for you. You need to explicitly define them as part of your migrations.

4.2.2.6 `:inverse_of`

The `:inverse_of` option specifies the name of the `belongs_to` association that is the inverse of this association. Does not work in combination with the `:through` or `:as` options.

```
class Supplier < ActiveRecord::Base
  has_one :account, inverse_of: :supplier
end

class Account < ActiveRecord::Base
  belongs_to :supplier, inverse_of: :account
end
```

4.2.2.7 `:primary_key`

By convention, Rails assumes that the column used to hold the primary key of this model is `id`. You can override this and explicitly specify the primary key with the `:primary_key` option.

4.2.2.8 `:source`

The `:source` option specifies the source association name for a `has_one :through` association.

4.2.2.9 `:source_type`

The `:source_type` option specifies the source association type for a `has_one :through` association that proceeds through a polymorphic association.

4.2.2.10 `:through`

The `:through` option specifies a join model through which to perform the query. `has_one :through`associations were discussed in detail earlier in this guide.

4.2.2.11 `:validate`

If you set the `:validate` option to `true`, then associated objects will be validated whenever you save this object. By default, this is `false`: associated objects will not be validated when this object is saved.

4.2.3 Scopes for `has_one`

There may be times when you wish to customize the query used by `has_one`. Such customizations can be achieved via a scope block. For example:

```
class Supplier < ActiveRecord::Base
  has_one :account, -> { where active: true }
end
```